Parul® University

Enterprise Programming (303105309)

Prof. Pirmohammad, Assistant Professor
Computer Science & Engineering

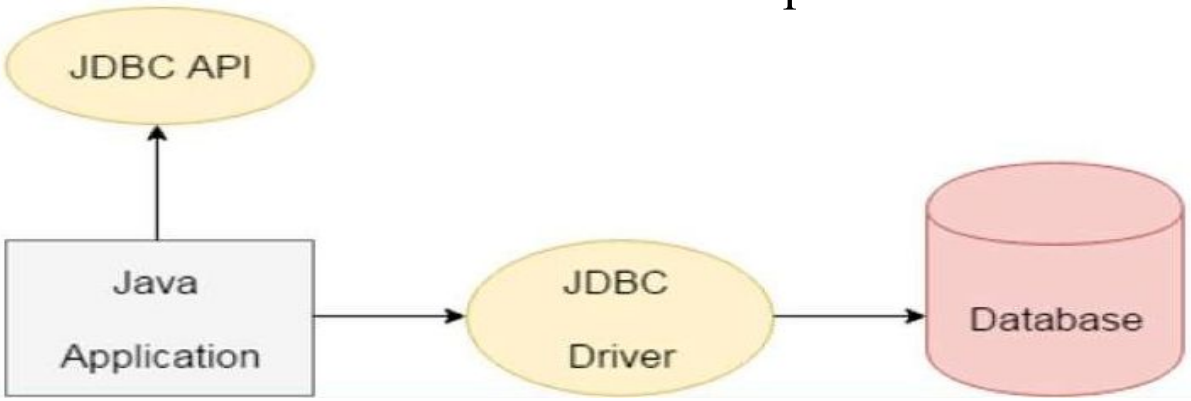
Parul®
University

DIGITAL LEARNING CONTENT

JDBC And Architecture of JDBC

Introduction to JDBC (Java Database Connectivity)

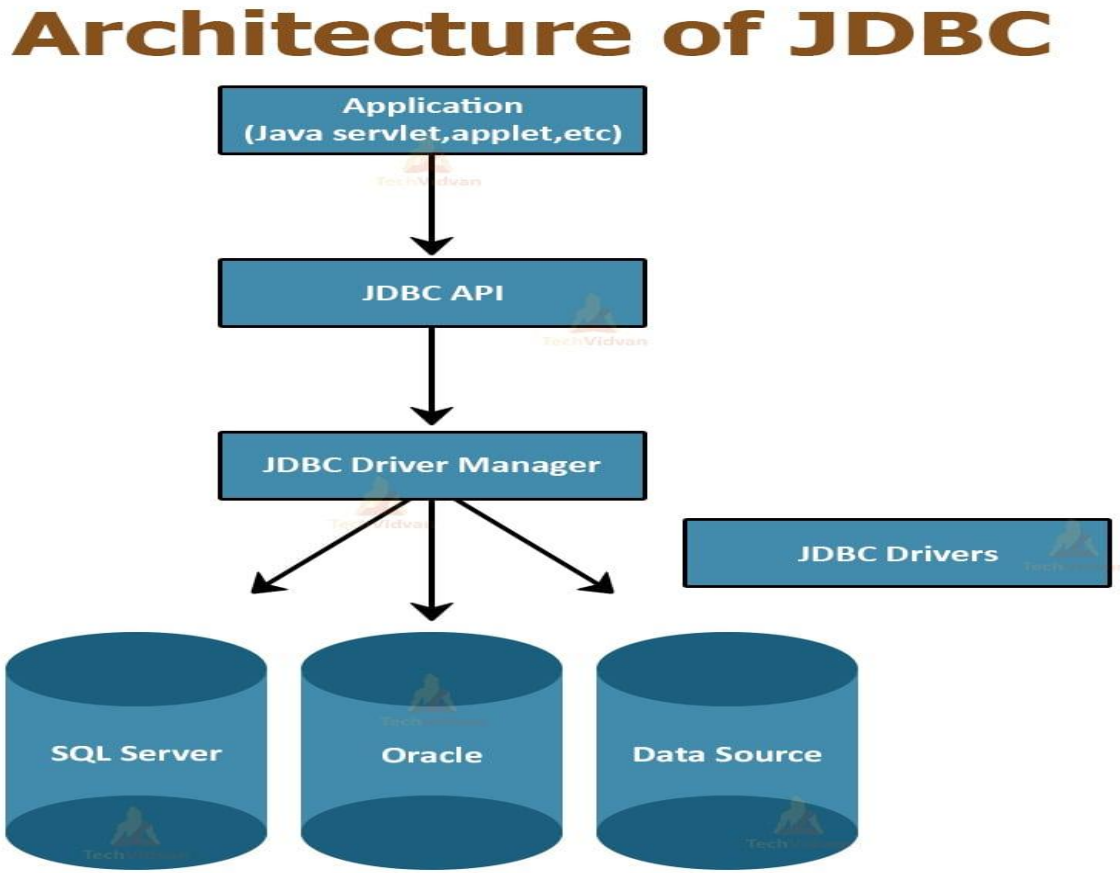
- ✔JDBC is a Java API to connect and execute the query with the database.
- ✔It is a specification from Sun Microsystems that provides an abstraction (API or Protocol) for Java applications to communicate with various databases such as Oracle, MS Access, Mysql, SQL server database, and many more.
- ✔It provides the language with Java database connectivity standards.
- ✔JDBC and database drivers work in tandem to access spreadsheets and databases.



Why Should We Use JDBC

- ✔Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).
- ✔We can use JDBC API to handle database using Java program and can perform the following activities:
  1. Connect to the database
  2. Execute queries and update statements to the database
  3. Retrieve the result received from the database.

Components of JDBC



Different Types of Architecture of JDBC

The architecture of the JDBC consists of two and three tiers model in order to access the given database.

Two-tier model: A java application communicates directly to the data source. The JDBC driver enables the communication between the application and the data source. When a user sends a query to the data source, the answers for those queries are sent back to the user in the form of results. The data source can be located on a different machine on a network to which a user is connected. This is known as a client/server configuration, where the user's machine acts as a client, and the machine has the data source running acts as the server.

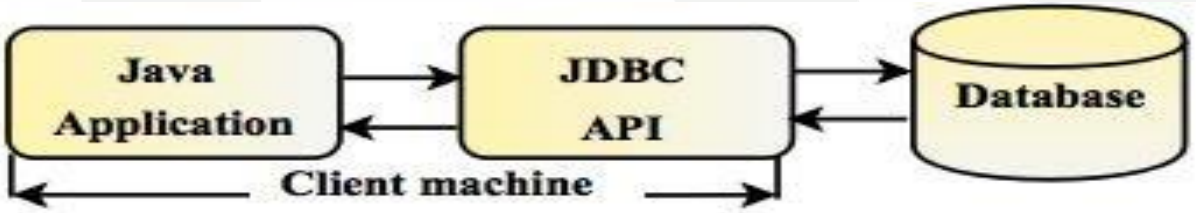


Fig: Two-tier Architecture of JDBC

Different Types of Architecture of JDBC

Three-tier model: In this, the user's queries are sent to middle-tier services, from which the commands are again sent to the data source. The results are sent back to the middle tier, and from there to the user.

This type of model is found very useful by management information system directors.

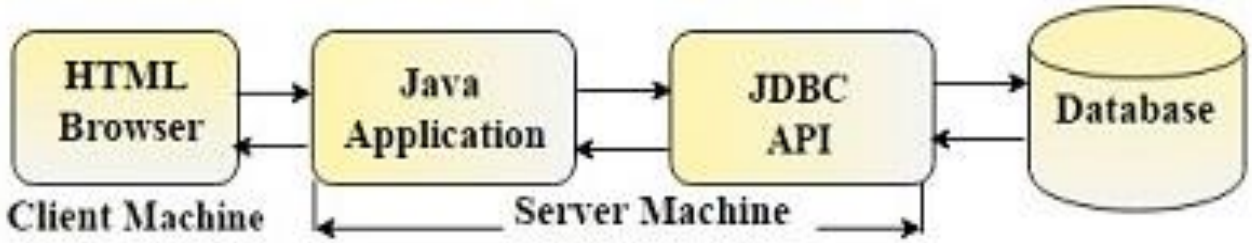


Fig: Three-tier Architecture of JDBC

Parul®
University

DIGITAL LEARNING CONTENT

JDBC Drivers

JDBC Drivers

- JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the database can understand.

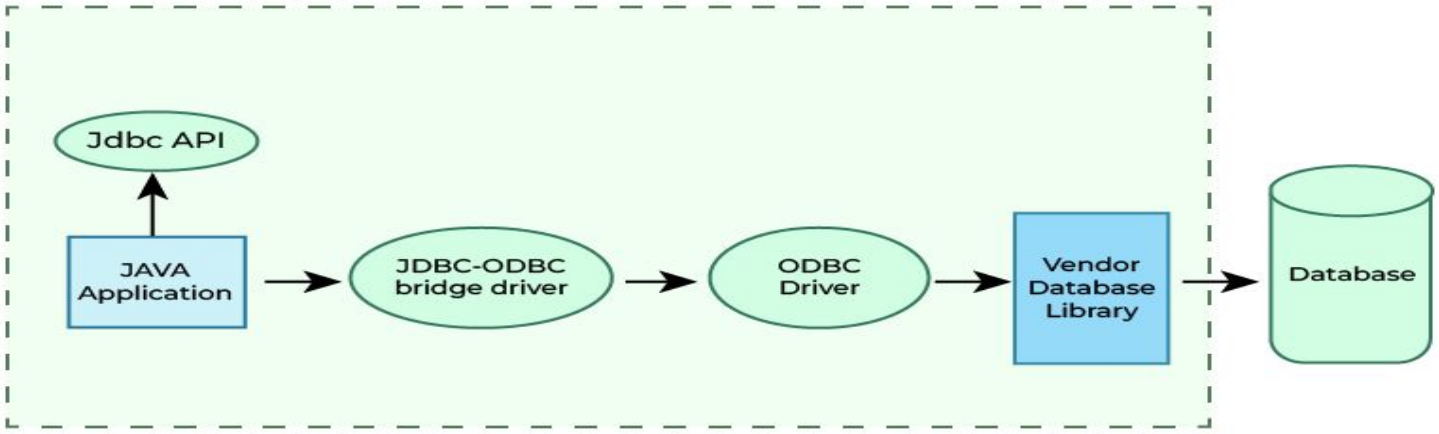
- There are 4 types of JDBC drivers:

1. Type-1 driver or JDBC-ODBC bridge driver
2. Type-2 driver or Native-API driver
3. Type-3 driver or Network Protocol driver
4. Type-4 driver or Thin driver

JDBC Drivers

Type-1 driver or JOBC-ODBC bridge driver

- It is an ODBC driver to connect with different databases.
- It converts JDBC method calls into ODBC function calls.
- It is a database-independent driver.
- No need to install it separately as built-in with JDK.



JDBC Drivers

Advantages:

- Built-in with JDK (no separate installation required).
- Database-independent.

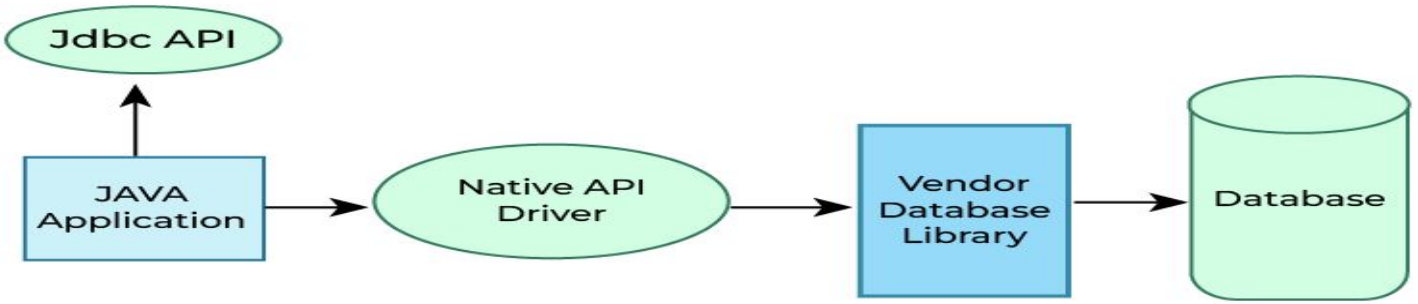
Disadvantages:

- Data transferred through this driver is not very secure.
- Requires ODBC bridge driver installation on individual client machines.
- Not written in Java, so it isn't portable

JDBC Drivers

Type-2 driver or Native-API driver

- It uses the client-side database libraries of the database for connectivity.
- It converts JDBC method calls into the native call of the database API.
- It needs their local API, for that data transfer is more secure as compared to Type-1.
- It needs to be installed individually for each client machine.



JDBC Drivers

Advantages:

- Better performance than the JDBC-ODBC bridge driver.
- More secure data transfer.

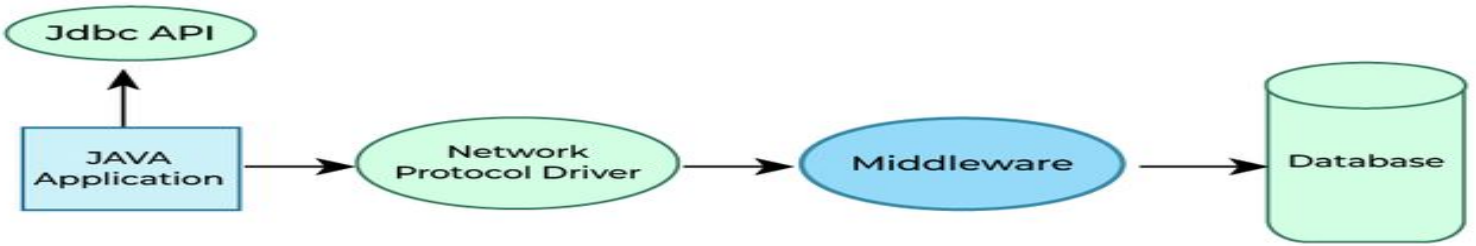
Disadvantages:

- Requires separate installation on client machines.
- Vendor client library must be installed on the client machine.
- Not fully written in Java (partially Java driver)

JDBC Drivers

Type-3 driver or Network Protocol driver

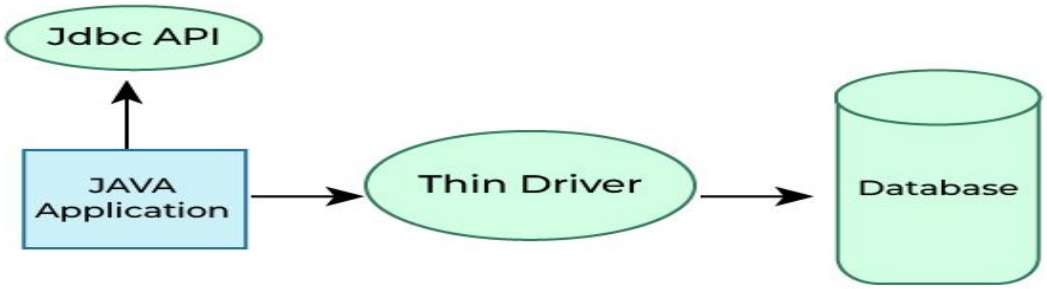
- It uses middleware (application server) that converts JDBC calls into vendor-specific database protocol.
- Drivers are present in a single server.
- It is a portable driver because fully written in Java.
- No client-side library is required because of application server can perform many tasks like auditing, load balancing, logging etc.
- Maintenance of Network Protocol driver becomes costly



JDBC Drivers

Type-4 driver or Thin driver

- It is also called a native protocol driver.
- It does not require any native database library, that is why it is also known as Thin Driver.
- It is a portable driver because fully written in java.



JDBC Drivers

Advantages:

- High performance.
- Secure data transfer.
- Portable (written entirely in Java).

Which Driver to Use When?

- If you are accessing **one type of database**, such as Oracle, Sybase, or IBM, the preferred driver type is type-4.
- If your Java application is **accessing multiple types of databases at the same time**, type 3 is the preferred driver.
- Type 2 drivers are useful in situations, where a **type 3 or type 4 driver is not available** yet for your database.
- The type 1 driver is not considered a deployment-level driver and is typically used for development and **testing purposes only**.

Parul® University

DIGITAL LEARNING CONTENT

Interfaces of JDBC API

- Driver interface
- Connection interface
- Statement interface
- Prepared Statement interface
- Callable statement interface
- Result Set interface
- Result Set MetaData interface
- Database MetaData interface
- Rowset interface

Classes of JDBC API

- Driver Manager class
- Blob class
- Clob class
- Types class

Class forName() method

- This forName() method of java.lang.Class class is used to get the instance of this class with the specified class name.
- This class name is specified as the string parameter.

Syntax

Public static Class<T> forName(String className) throws ClassNotFoundException

Example

class.forName(“com.mysql.cj.jdbc.Driver”) -> For MySql Connectivity

class.forName(“oracle.jdbc.driver.OracleDriver”) -> For Oracle connectivity

Driver Manager

- The DriverManager class is the main component of JDBC API.
- The DriverManager class acts as an interface between users and drivers.
- It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver.
- It also contains all the appropriate methods to register and deregister the database driver class and to create a connection between a Java application and the database.

Methods:

public static void registerDriver(Driver driver)
public static void deregisterDriver(Driver driver)
public static Connection getConnection(String Url)
public static Connection getConnection(String Url, String username, String password)

Example

String connectionUrl = “jdbc:mysql://localhost:3307/**dbname**”
Connection conn = DriverManager.getConnection(connectionUrl, "root", "password");

Connection Interface

- Connection interface is used for creating the session between the application and the database.
- This interface contains Statement, PreparedStatement and DatabaseMetaData.

Methods

public Statement createStatement()

public Statement createStatement(int resultSetType,int resultSetConcurrency)

public void setAutoCommit(boolean status)

public void commit()

public void rollback()

public void close()

Statement Interface

The Statement interface is used for executing queries using the database.
This interface is a factory of ResultSet.
It is used to get the Object of ResultSet.

Methods

public ResultSet executeQuery(String sql)
public int executeUpdate(String sql)
public boolean execute(String sql)
public int[] executeBatch()

Example

```
Statement stmt = conn.createStatement();
ResultSet result = stmt.executeQuery(“Select * From students”);
```

ResultSet Interface

- The ResultSet Interface is used for maintaining the pointer to a row of a table. Initially, cursor points to before the first row.
- The object can be moved forward as well as backward direction using TYPE _SCROLL_INSENSITIVE or TYPE _SCROLL_SENSITIVE in createStatement(int,int).
- TYPE _SCROLL_INSENSITIVE - Changes done in the database are not reflected in the ResultSet.
- TYPE _SCROLL_SENSITIVE - Changes done in the database are reflected in the Resultset.
- CONCUR_UPDATABLE -— This type of result set is updatable.
- CONCUR_ READONLY - This type of result set is not updatable
- **For Example**

Statement stmt = con.createStatement(ResultSet. TYPE _SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);

Example

ResultSet result = stmt.executeQuery(“Select * From students”);

Example

```
try {
    Class. forName("com.mysql.cj.jdbc.Driver") ;
    conn =DriverManager.getConnection("jdbc:mysql://localhost:3306/myData","root",
"pwd" ) ;
    Statement stmt = conn.createStatement();

    ResultSet result = stmt.executeQuery(“select * from employees”);
    while(result.next()) {
    System.out.println(result.getInt(1) + "\t" + result.getString(2) +" "+
result.getString(3));
    }
    conn.close();
}
catch (Exception e) {
     System.out.println("my Error :" + e.getMessage());
}
```

PreparedStatement Interface

- PreparedStatement interface is a subinterface of Statement.
- It is mainly used for the parameterized queries. A question mark (?) is passed for the values.
- The values to this question marks will be set by the PreparedStatement.

Example

PreparedStatement stmt = conn.prepareStatement(“insert into Employees values(?,?, ?)”);
stmt.setint(1, 1001)
stmt.setString(2, “Mahesh”)
stmt.setint(3, “Shah”)
int result = stmt.executeUpdate();

ResultSetMetaData Interface

ResultSetMetaData interface is used to get metadata from the ResultSet object.
Metadata are the data about data.

Methods

public int getColumnCount() throws SQLException
public String getColumnName(int index) throws SQLException
public String getColumnTypeName(int index) throws SQLException
public String getTableName(int index) throws SQLException

Example

```
ResultSetMetaData resmd = result.getMetaData();
System.out.printIn("Total columns: “ + resmd.getColumnCount());
System.out.printIn("Column Name of 1st column: “ + resmd.getColumnName(1));
System.out.printIn("Column Type Name of ist column: “ +resmd.getColumnTypeName(1));
```

DatabaseMetaData Interface

- DatabaseMetaData interface provides methods to get meta data of a database such as database product name, database product version, driver name, name of total number of tables, name of total number of views etc.

Methods

public String getDriverName() throws SQLException
public String getDriverVersion() throws SQLException
public String getUserName() throws SQLException
public String getDatabaseProductName() throws SQLException
public String getDatabaseProductVersion() throws SQLException
public ResultSet getTables(String catalog, String schemaPattern, String tableNamePattern,String[] types) throws SQLException

Example

```
DatabaseMetaData dbmd=con.getMetaData();
System.out.printiln("Driver Name: "+dbmd.getDriverName());
System.out.println("Driver Version: "+dbmd.getDriverVersion());
System.out.printiln(""“UserName: "+dbmd.getUserName());
System.out.println("Database Product Name: "+dbmd.get DatabaseProduct
Name());
System.out.println("Database Product Version: "+dbmd.getDatabase Product
Version());
```

Parul® University

RowSet Interface

- AJDBC RowSet facilitates a mechanism to keep the data in tabular form.
- It is the wrapper of ResultSet.
- AJDBC RowSet object holds tabular data in a style that makes it more adaptable and simpler to use than a result set.
- The implementation classes of the RowSet interface are as follows:
  1. JdbcRowSet
  2. CachedRowSet
  3. WebRowSet
  4. JoinRowSet
  5. FilteredRowSet

JdbcRowSet

- It basically acts as a wrapper around the ResultSet object with some additionalfunctionality.
- It is only connected RowSet in the family.
- The primary advantage of using JdbcRowSet is that it enables the ResultSet object tobe used as a JavaBeans component.
- It is scrollable and updatable capabilities to the ResultSet object.

CachedRowSet

- CachedRowSet object is unique because it can operate without being connected toits data source. We call this a “disconnected RowSet object”.
- it caches its data in memory so that it can operate on its own data instead of thedata stored in a database.

WebRowSet

- It extends CachedRowSet capabilities but is very special in the sense that in additionto providing all the features of CachedRowSet, it can read and write XML document.
- it can write itself to an XML document and can also read that XML document toconvert itself back to a WebRowSet.
- it is mainly used in enterprise application scenario or in web service communication.

FilteredRowSet

- It is Jonrowe an extension of WebRowSet.
- filtering data based on criteria to fetch selected rows from the data source so that we can work with the relevant data.
- It is something like using the WHERE clause without writing an SQL.

Image source : Google

Join RowSet Interface

- It provides capabilities of WebRowSet and CachedRowSet, we can perform a SQL JOIN operation without connecting to a data source.
- It enables us to create SQL JOIN between RowSet objects and Related records from different RowSets can be combined to form this RowSet object.

Parul®
University

Jdbc with Oracle

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- Register the Driver class
- Create connection
- Create statement
- Execute queries
- Close connection

1) Register the driver class

The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax of forName() method

public static void forName(String className)**throws** ClassNotFound Exception

Example to register the OracleDriver class

Here, Java program is loading oracle driver to esteblish database connection.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

2) Create the connection

The **getConnection()** method of DriverManager class is used to establish connection with the database.

Syntax of getConnection() method

1) **public static** Connection getConnection(String url)**throws** SQLException
2) **public static** Connection getConnection(String url,String name,String password) **throws** SQLException

Example to establish connection with the Oracle database

```
Connection con=DriverManager.getConnection(  "jdbc:oracle:thin:@localhost:1521
:xe","system","password");
```

3) Create the Statement object

The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

Syntax of createStatement() method

public Statement createStatement()**throws** SQLException

Example to create the statement object

Statement stmt=con.createStatement();

4) Execute the query

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method

public ResultSet executeQuery(String sql)**throws** SQLException

Example to execute query

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

5) Close the connection

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method

```
public void close()throws SQLException
```

Example to close connection

```
con.close();
```

Parul®
University

DIGITAL LEARNING CONTENT

JDBC with Oracle and Mysql

Java Database Connectivity with Oracle

To connect java application with the oracle database, we need to follow 5 following steps. In this example, we are using Oracle 10g as the database. So we need to know following information for the oracle database:

1. **Driver class:** The driver class for the oracle database is **oracle.jdbc.driver.OracleDriver**.
2. **Connection URL:** The connection URL for the oracle10G database is **jdbc:oracle:thin:@localhost:1521:xe** where jdbc is the API, oracle is the database, thin is the driver, localhost is the server name on which oracle is running, we may also use IP address, 1521 is the port number and XE is the Oracle service name. You may get all these information from the tnsnames.ora file.
3. **Username:** The default username for the oracle database is **system**.
4. **Password:** It is the password given by the user at the time of installing the oracle database.

Java Database Connectivity with Oracle

Create a Table

Before establishing connection, let's first create a table in oracle database. Following is the SQL query to create a table.

```
create table emp(id number(10),name varchar2(40),age number(3));
```

Example to Connect Java Application with Oracle database

In this example, we are connecting to an Oracle database and getting data from **emp** table. Here, **system** and **oracle** are the username and password of the Oracle database.

Example to Connect Java Application with Oracle database

```
import java.sql.*;
class OracleCon{
public static void main(String args[]){
try{
//step1 load the driver class
Class.forName("oracle.jdbc.driver.OracleDriver");
//step2 create  the connection object
Connection con=DriverManager.getConnection(
"jdbc:oracle:thin:@localhost:1521:xe","system","oracle");
//step3 create the statement object
Statement stmt=con.createStatement();
//step4 execute query
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getString(3));
//step5 close the connection object
con.close();
}
catch(Exception e){ System.out.println(e);}
}
}
```

Example to Connect Java Application with MySQL database

1. **Driver class:** The driver class for the mysql database is **com.mysql.jdbc.Driver**.
2. **Connection URL:** The connection URL for the mysql database is **jdbc:mysql://localhost:3306/sonoo** where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and sonoo is the database name. we may use any database, in such case, we need to replace the sonoo with our database name.
3. **Username:** The default username for the mysql database is **root**.
4. **Password:** It is the password given by the user at the time of installing the mysql database.

Example to Connect Java Application with MySQL database

```
import java.sql.*;
class MysqlCon{
public static void main(String args[]){
try{
Class.forName("com.mysql.jdbc.Driver");
Connection con=DriverManager.getConnection(
"jdbc:mysql://localhost:3306/sonoo","root","root");
//here sonoo is databse name, root is username and password
Statement stmt=con.createStatement();
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getString(3));
con.close();
}
catch(Exception e){ System.out.println(e);
}
}
}
```

Parul®
University

DIGITAL LEARNING CONTENT

Maven Integration with Eclipse

What is Maven?

Definition: Maven is a build automation and project management tool for Java projects.
It simplifies the process of building, testing, and deploying Java applications by managing project dependencies and providing a standardized project structure.

Purpose: Manages project dependencies, enforces a standard project structure, and automates the build process (compile, test, package).

Maven's Objectives

- Making the build process easy
- Providing a uniform build system
- Providing quality project information
- Encouraging better development practices

Benefits of Using Maven:

Automatic Dependency Management:

- Handles downloading and updating libraries.
- Ensures the right versions of dependencies are used.

Standardized Project Structure:

- Provides a consistent layout for projects.
- Makes it easier to navigate and maintain code.

Build Automation:

- Automates tasks such as compiling code, running tests, and packaging applications.
- Supports different build profiles for development, testing, and production.

Integration with IDEs:

- Seamlessly integrates with Eclipse via the Maven Integration (m2e) plugin.
- Enhances productivity by managing Maven tasks within the IDE.

Benefits of Using Maven:

Plugin Ecosystem:

- Extends Maven's functionality through a wide range of plugins.
- Supports additional tasks like code quality checks and documentation generation.

Improved Collaboration:

- Ensures a unified build process for all team members.
- Simplifies dependency and configuration management across different environments.

How to Integrate Maven with Eclipse:

Install Maven in Eclipse:

- Go to Help > Eclipse Marketplace.
- Search for "Maven Integration for Eclipse" (m2e) and install it.

Create a Maven Project:

- Click File > New > Other.
- Select Maven Project and follow the prompts to set up your project.

Add Dependencies:

- Open the pom.xml file in your Maven project.
- Add required dependencies inside the <dependencies> tag. Maven will handle downloading and managing these libraries.

Build and Run the Project:

- Right-click your project.
- Select Run As > Maven build.
- Choose build goals (e.g., clean, install) to compile, test, and package your application.

Key Terms:

- **POM (Project Object Model):** The pom.xml file is the core of a Maven project, containing configuration details and dependencies.
- **Dependency:** External libraries required by your project.
- **Build Lifecycle:** The sequence of phases that define the build process, including clean, compile, test, package, and install.
- **Plugin:** An extension that adds specific functionalities to Maven.

Parul®
University

DIGITAL LEARNING CONTENT

POM.xml

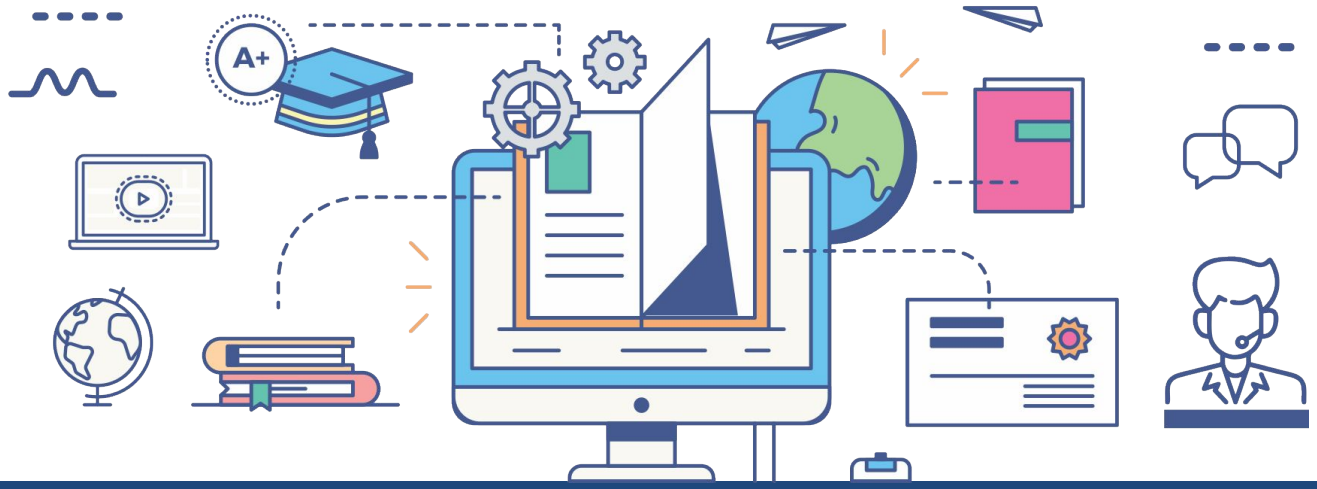
What is POM file?

- A Project Object Model or POM is the fundamental unit of work in Maven.
- It is an XML file that contains information about the project and configuration details used by Maven to import dependencies and to build the project.
- Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, then executes the goal.
- Few things to notice are:
  Default repo is maven repository.
  Default execution goal is ‘jar’.
  Default source code location is src/main/java.
  Default test code location is src/test/java.

Example

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0"xmins:xsi="ht
tp://www.w3.org/2001/XMLSchema-instance"xSi:schemaLocation="http://mav
en.apache.org/POM/4.0.0http: //maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<packaging>pom</packaging>
<parent>
<grouplid>org.springframework.boot</groupId>
<artifactid>spring-boot-starter-parent</artifactId>
<version>2.2.0.M5</version>
</parent>
</project>
```

DIGITAL LEARNING CONTENT



Parul® University

www.paruluniversity.ac.in